K.R. MANGALAM UNIVERSITY

SCHOOL OF ENGINEERING &TECHNOLOGY

Department of Computer Science & Technology

Assignment Number: 1

Unit Title: Foundations & Algorithmic Analysis

Assignment Title: Algorithmic Efficiency & Recursion Toolkit

Course Code: ETCCDS202

Course Name: Data Structures

Faculty: Mrs. Neetu Chauhan

Student Name: Jiya Gupta

Roll Number: 2501730209

Section: A

Programme: B. Tech CSE (Specialization in AI & ML)

PART-A

What is an ADT?

An Abstract Data Type (ADT) is a logical model that defines a data structure by its data and allowed operations, without describing the internal implementation. It focuses on what operations can be performed, not how they are implemented.
Examples include Stack, Queue, and List ADTs .

Why stack operations are usually O(1)?

Stack operations such as push, pop, and peek work only on the top element of the stack. They do not require searching or traversing other elements. Therefore, each operation takes constant time O(1) regardless of the stack size.

PART-B

Factorial (Recursive)

Time Complexity

$$T(n) = T(n-1) + O(1)$$

Each call makes one recursive call and one multiplication.
Total number of calls = n

- Big-O: $O(n)$
- Big-Ω: $\Omega(n)$
- Big-Θ: $\Theta(n)$

Space Complexity

- Recursive depth = n
- $O(n)$, $\Theta(n)$

Fibonacci (Naive Recursive)

Time Complexity

$T(n) = T(n-1) + T(n-2) + O(1)$

Each call creates two more calls, forming a recursion tree.
Number of calls grows exponentially.

- Big-O: $O(2^n)$
- Big-Ω: $\Omega(2^n)$
- Big-Θ: $\Theta(2^n)$

Space Complexity

Maximum recursion depth = n

- $O(n)$, $\Theta(n)$

Fibonacci (Memorized)

Each Fibonacci number is calculated only once and stored.

Time Complexity

Total computations = n

- Big-O: $O(n)$
- Big-Ω: $\Omega(n)$
- Big-Θ: $\Theta(n)$

Space Complexity

Memo storage + recursion stack

- $O(n)$, $\Theta(n)$

Why Naive Fibonacci is Slow for Large n?

- It recomputes the same subproblems many times.

- Number of recursive calls grows exponentially.
- Running time increases very fast as n increases.
- Hence, it becomes inefficient for large n.

PART-C

Why the number of moves becomes 2n−12^n - 12n−1

To move n disks, the algorithm follows these steps:

1. Move n − 1 disks from source to auxiliary.
2. Move the largest disk from source to destination.
3. Move n − 1 disks from auxiliary to destination.

This gives the recurrence relation:

T(n)=2T(n−1)+1T(n) = 2T(n-1) + 1T(n)=2T(n−1)+1

Solving this recurrence results in:

T(n)=2n−1T(n) = 2^n - 1T(n)=2n−1

So the minimum number of moves required to solve Tower of Hanoi with n disks is

2n−12^n - 12n−1.

Example:

n	Moves
1	1
2	3
3	7
4	15

Time Complexity

$O(2^n)$

The number of recursive calls grows exponentially because each step generates two recursive calls for n−1n-1n−1 disks.

Space Complexity

O(n)

The recursion depth is n, because the function keeps calling itself with n−1 disks until the base case is reached.
Each recursive call is stored in the call stack, resulting in O(n) space complexity.

PART-D

Binary search divides the array into two halves at each step and continues searching in only one half.

Recurrence Relation
$T(n) = T(n/2) + O(1)$

- $T(n/2) \rightarrow$ search in half of the array
- $O(1) \rightarrow$ constant time comparison After each step, problem size becomes half:
  $n \rightarrow n/2 \rightarrow n/4 \rightarrow n/8 \rightarrow \ldots \rightarrow 1$

  Number of times we can divide n by 2 until it becomes 1 is:
  $\log_2(n)$
  Therefore:
- Time Complexity = $O(\log n)$
- Big-Θ = $\Theta(\log n)$
- Big-Ω = $\Omega(1)$ (best case)

PART-E

1. Binary Search – Divide and Conquer

Binary search follows the Divide and Conquer paradigm.
The algorithm divides the sorted array into two halves by checking the middle element.
If the key is smaller than the middle element, the search continues in the left half; otherwise it continues in the right half.
This process keeps dividing the problem into smaller subproblems until the element is found or the search space becomes empty.

2. Fibonacci with Memoization – Dynamic Programming

Memoized Fibonacci follows the Dynamic Programming paradigm.
In the naive recursive Fibonacci, the same values are computed many times, which increases the number of recursive calls.
Memoization stores previously calculated Fibonacci values in a table (memory).
When the same value is required again, it is taken from memory instead of recomputing it, which greatly improves efficiency.

3. Tower of Hanoi – Recursion

Tower of Hanoi is a classic example of recursion.
The problem is solved by breaking it into smaller subproblems.

Base case:
If there is only one disk, move it directly from the source rod to the destination rod.

Recursive idea:
To move n disks, first move $n-1$ disks to the auxiliary rod, then move the largest disk to the destination rod, and finally move the $n-1$ disks from auxiliary to destination.

This repeated reduction of the problem makes recursion suitable for solving Tower of Hanoi.

Real-Life Examples of Algorithm Paradigms

Divide and Conquer

Searching for a word in a dictionary.
You open the dictionary near the middle and decide whether to search the left or right section depending on the alphabetical order.

Dynamic Programming

Finding the shortest route in Google Maps.
The system stores previously computed shortest paths and uses them to determine the best route efficiently.

Greedy Algorithm

Selecting the minimum number of coins to make a certain amount of money.
At each step, the algorithm chooses the largest possible coin value, which is the locally optimal choice.

Conclusion

This report explored fundamental concepts of data structures, recursion, and algorithm analysis through the implementation of the Algorithmic Efficiency & Recursion Toolkit (AERT). The Stack ADT demonstrated how abstract data types define operations such as push and pop while keeping the implementation independent. Recursive algorithms like factorial, Fibonacci, and Tower of Hanoi showed how complex problems can be solved by breaking them into smaller sub problems.

The comparison between naive Fibonacci and memoized Fibonacci highlighted how optimization techniques like Dynamic Programming reduce redundant computations and improve efficiency. The binary search algorithm demonstrated the effectiveness of the Divide and Conquer paradigm, reducing search time significantly compared to linear search.

Overall, this assignment helped in understanding how different algorithm paradigms and recursion techniques are applied to solve problems efficiently and how time and space complexity analysis is important in evaluating algorithm performance.